

Hash Table

Linear Probing

HashTable

```
package hashTable.linearProbing;

public class HashTable implements Position{
    private Object[] arr;
    private int capacidade, size;

    public HashTable(int capacidade){
        this.capacidade = capacidade;
        this.size = 0;
        arr = new Object[capacidade];
    }

    public void insert(Object elemento, int key) {

        double alfa = (double) size / capacidade; //diz o quanto encheu

        if (alfa > 0.5){
            grow();
        }

        Item novo_item = new Item(elemento, key); //esse é o novo elemento com chave / dici

        int indice_elemento = novo_item.getKey() % capacidade; //indice do elemento com cha

        while (arr[indice_elemento] != null ){
            //coloca o circular para ele andar
            indice_elemento = (indice_elemento + 1) % capacidade;
        }

        arr[indice_elemento] = novo_item;

        size++;
    }

    public Object remove(int key) throws InvalidHashTable{
        if (isEmpty()){
            throw new InvalidHashTable("Nenhum item para remover, está vazio!");
        }
    }
}
```

```

    int indice = key % capacidade; //primeiro pego a chave e depois irei pegar o que es

    Object removido = null;

    while (arr[indice] != null){ //enquanto tiver indice

        Item armazenado = (Item) arr[indice]; //o que já existe da chave e elemento, ace

        if (key != armazenado.getKey()){
            indice = (indice + 1) % capacidade; //anda o indice!
        } else{
            removido = armazenado.getElement(); //armazena
            arr[indice] = null; //ele tira o item do indice armazenado
            break;
        }
    }

    if (removido == null){
        throw new InvalidHashTable("não achou a chave para remover");
    }

    size--;
    return removido;
}

public Object find(int key) throws InvalidHashTable{
    if (isEmpty()){
        throw new InvalidHashTable("Nenhum item para remover, está vazio!");
    }

    Object achado = null;

    int indice = key % capacidade; //lembre-se que a posicao sempre é int

    while (arr[indice] != null){

        Item armazenado = (Item) arr[indice];

        if (key != armazenado.getKey()){
            indice = (indice + 1) % capacidade; //não esqueça do + 1 para poder andar, s
        } else {
            achado = armazenado.getElement();
            break;
        }
    }
}

```

```

        if (achado == null){
            throw new InvalidHashTable("Não encontrou a chave!");
        }

        return achado;
    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public void grow(){
        int nova_capacidade = capacidade * 2;
        Object[] novo_arr = new Object[nova_capacidade];

        for (int i = 0; i < capacidade; i++){
            if (arr[i] != null){
                Item velho_armazenado = (Item) arr[i];
                int novo_indice = velho_armazenado.getKey() % nova_capacidade;

                while (novo_arr[novo_indice] != null){
                    novo_indice = (novo_indice + 1) % nova_capacidade;
                }

                novo_arr[novo_indice] = velho_armazenado;
            }
        }

        capacidade = nova_capacidade;
        arr = novo_arr;
    }
}

```

Teste do Linear

```

package hashTable.linearProbing;

public class Teste {
    public static void main(String[] args) {

        HashTable tabela = new HashTable(4);
    }
}

```

```

// INSERT
tabela.insert("A", 1);
tabela.insert("B", 2);
tabela.insert("C", 3);
tabela.insert("D", 4);

// isso deve forçar grow()
tabela.insert("E", 5);

System.out.println("Tamanho: " + tabela.size());

// FIND
try {
    System.out.println("Find 3: " + tabela.find(3)); // C
    System.out.println("Find 5: " + tabela.find(5)); // E
} catch (Exception e) {
    System.out.println(e.getMessage());
}

// REMOVE
try {
    System.out.println("Remove 2: " + tabela.remove(2)); // B
    System.out.println("Remove 1: " + tabela.remove(1)); // A
} catch (Exception e) {
    System.out.println(e.getMessage());
}

// TESTE APÓS REMOÇÃO
try {
    System.out.println("Find 2: " + tabela.find(2)); // deve lançar exceção
} catch (Exception e) {
    System.out.println(e.getMessage());
}

System.out.println("Final size: " + tabela.size());
}
}

```

Hash Duplo

HashTable

```

package hashTable.hashDuplo;

public class HashTable implements Position{

```

```

private Object[] arr;
private int capacidade, size;
private int q;

public HashTable(int capacidade){
    this.capacidade = capacidade;
    this.size = 0;
    arr = new Object[capacidade];
    this.q = 7;
}

public void insert(Object elemento, int key) {

    double alfa = (double) size / capacidade;

    if (alfa > 0.5){
        grow();
    }

    Item novo_item = new Item(elemento, key);

    int indice_elemento = novo_item.getKey() % capacidade;
    int pulo = q - (novo_item.getKey() % q);

    while (arr[indice_elemento] != null){
        indice_elemento = (indice_elemento + pulo) % capacidade;
    }

    arr[indice_elemento] = novo_item;

    size++;
}

public Object remove(int key) throws InvalidHashTable{
    if (isEmpty()){
        throw new InvalidHashTable("Nenhum item para remover, está vazio!");
    }

    int indice = key % capacidade;
    int pulo = q - (key % q);

    Object removido = null;

    while (arr[indice] != null){

```

```

        Item armazenado = (Item) arr[indice];

        if (key != armazenado.getKey()){
            indice = (indice + pulo) % capacidade;
        } else{
            removido = armazenado.getElement();
            arr[indice] = null;
            break;
        }
    }

    if (removido == null){
        throw new InvalidHashTable("não achou a chave para remover");
    }

    size--;
    return removido;
}

public Object find(int key) throws InvalidHashTable{
    if (isEmpty()){
        throw new InvalidHashTable("Nenhum item para remover, está vazio!");
    }

    Object achado = null;

    int indice = key % capacidade;
    int pulo = q - (key % q);

    while (arr[indice] != null){

        Item armazenado = (Item) arr[indice];

        if (key != armazenado.getKey()){
            indice = (indice + pulo) % capacidade;
        } else {
            achado = armazenado.getElement();
            break;
        }
    }

    if (achado == null){
        throw new InvalidHashTable("Não encontrou a chave!");
    }

    return achado;
}

```

```

    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public void grow(){
        int nova_capacidade = capacidade * 2;
        Object[] novo_arr = new Object[nova_capacidade];

        for (int i = 0; i < capacidade; i++){
            if (arr[i] != null){
                Item velho_armazenado = (Item) arr[i];
                int novo_indice = velho_armazenado.getKey() % nova_capacidade;
                int pulo = q - (velho_armazenado.getKey() % q);

                while (novo_arr[novo_indice] != null){
                    novo_indice = (novo_indice + pulo) % nova_capacidade;
                }

                novo_arr[novo_indice] = velho_armazenado;
            }
        }

        capacidade = nova_capacidade;
        arr = novo_arr;
    }
}

```

Teste do Linear

```

package hashTable.hashDuplo;

public class Teste {
    public static void main(String[] args) {

        HashTable ht = new HashTable(5);

        // INSERT
    }
}

```

```

        ht.insert("A", 1);
        ht.insert("B", 2);
        ht.insert("C", 3);
        ht.insert("D", 4);
        ht.insert("E", 5);

        System.out.println("Size após insert: " + ht.size());

        // FIND
        try {
            System.out.println("Find 3: " + ht.find(3));
            System.out.println("Find 5: " + ht.find(5));
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        // REMOVE
        try {
            System.out.println("Remove 2: " + ht.remove(2));
            System.out.println("Remove 1: " + ht.remove(1));
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        // TESTE DE BUSCA APÓS REMOÇÃO
        try {
            System.out.println("Find 2: " + ht.find(2));
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Final size: " + ht.size());
    }
}

```

Padrão Geral

InvalidHashTable

```

package hashTable.hashDuplo;

public class InvalidHashTable extends RuntimeException{
    public InvalidHashTable(String err){
        super(err);
    }
}

```


Item

```
package hashTable.hashDuplo;

public class Item{
    private Object elemento;
    private int key;

    public Item(Object elemento, int key){
        this.elemento = elemento;
        this.key = key;
    }

    public Object getElement(){
        return elemento;
    }

    public int getKey(){
        return key;
    }

    public void setElement(Object elemento){
        this.elemento = elemento;
    }

    public void setKey(int key){
        this.key = key;
    }
}
```

Position

```
package hashTable.hashDuplo;

public interface Position {
    void insert(Object elemento, int key);
    Object find(int key) throws InvalidHashTable;
    Object remove(int key) throws InvalidHashTable;
    boolean isEmpty();
    int size();
    void grow();
}
```